

COGNOMS:

NOM:

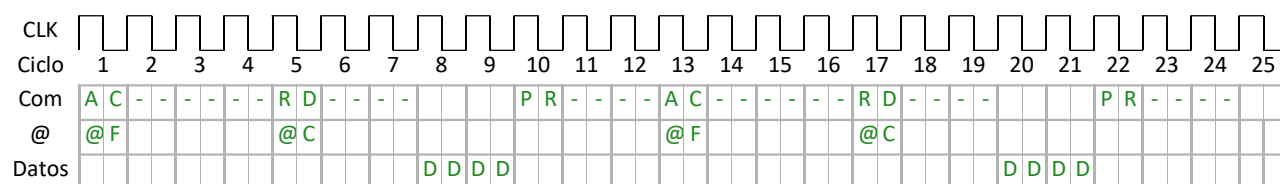
IMPORTANTE leer atentamente antes de empezar el examen: Escriba los apellidos y el nombre antes de empezar el examen. Escriba un solo carácter por recuadro, en mayúsculas y lo más claramente posible. Es importante que no haya tachones ni borrones y que cada carácter quede enmarcado dentro de su recuadro sin llegar a tocar los bordes. Use un único cuadro en blanco para separar los apellidos y nombres compuestos si es el caso. No escriba fuera de los recuadros.

Problema 1. (2.4 puntos)

Una **CPU** con arquitectura **VLIW** está conectada a una cache de instrucciones y una cache de datos. El conjunto formado por **CPU+caches** esta conectado a una memoria principal formada por un único módulo DIMM estándar de 16 GBytes. Este DIMM tiene 8 chips de memoria **DDR-SDRAM (Double Data Rate Synchronous DRAM)** de 1 byte de ancho cada uno. El DIMM esta configurado para leer/escribir ráfagas de 32 bytes (justo el tamaño de bloque de las caches). La latencia de fila es de 4 ciclos, la latencia de columna de 3 ciclos y la latencia de precarga de 3 ciclos.

En el siguiente cronograma, indica la ocupación de los distintos recursos de la memoria DDR: bus de datos, bus de direcciones y bus de comandos. Supondremos que no hay ninguna página de DRAM abierta y al acabar la secuencia de accesos todas las paginas deben quedar cerradas. El controlador de memoria envía los comandos necesarios a la DDR-SDRAM de forma que los bloques sean transferidos lo más rápidamente posible y se maximice el ancho de banda. El objetivo es minimizar el tiempo total.

a) **Rellena** el cronograma para la lectura de 2 bloques de 32 bytes ubicados en páginas distintas de un mismo banco.



En este procesador VLIW ejecutamos el siguiente fragmento de código:

```
for (i=0; i<N; i=i+1)
    suma = suma + v[i]; // v[i] es un vector de doubles (8 bytes)
```

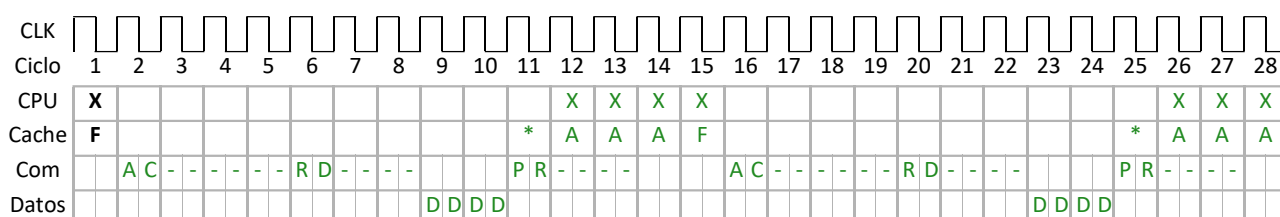
El código está almacenado en la cache de instrucciones, por lo que no habrá fallos de instrucción. Las variables **i**, **N** y **suma** están asignadas a registros del procesador, y la cache de datos está inicialmente vacía.

Una versión compilada de este código ha generado 5 operaciones para implementar el bucle. Mediante la técnica de *software pipelining*, se ha logrado compactar estas 5 operaciones en **una única instrucción VLIW**. Por lo tanto, en ausencia de fallos de cache, una iteración completa del bucle se ejecuta en **un único ciclo de reloj**.

En los siguientes cronogramas ten en cuenta estas consideraciones:

- **CPU:** Ejecuta una instrucción VLIW por ciclo (márcalo con **X**). Ante un fallo de cache de datos, la CPU se detiene (stall) en el ciclo siguiente y no reanuda la ejecución hasta que el bloque esté disponible.
- **Cache:** Indica los aciertos con **A** y los fallos con **F**. La carga del bloque en la cache se realiza progresivamente durante la transferencia de datos por parte de la memoria DDR. Debes marcar con ***** el ciclo en el que el bloque es entregado a la CPU, el ciclo inmediatamente posterior a su transferencia desde la DDR.
- **Memoria:** El controlador reacciona al fallo generando un acceso a la memoria DDR en el ciclo inmediatamente posterior al fallo. Por simplicidad, solo indicaremos la ocupación del bus de comandos y el bus de datos.
- Rellena todos los eventos correspondientes a las primeras iteraciones del bucle hasta el ciclo 28.

b) **Rellena** el siguiente cronograma suponiendo que tenemos un controlador sencillo que sigue una política de abrir y cerrar página en cada acceso a la DDR (**close-page**). El primer ciclo ya ha sido completado como guía.



Sabemos que el vector $\mathbf{v}$ esta alineado con el inicio de una página de DRAM y que los bloques de memoria están almacenados consecutivamente dentro de una misma página.

Para mejorar el rendimiento del sistema, se optimiza el controlador de memoria implementando una política de **página abierta (open-page)**, la cual mantiene la fila de la DRAM activa mientras se sucedan accesos a la misma página.

- c) **Rellena** el siguiente cronograma para este nuevo controlador (open-page). Asume que para el primer acceso al vector $\mathbf{v}$, la página de la DDR ya se encuentra abierta.

CLK																												
Ciclo	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
CPU	X							X	X	X	X							X	X	X	X							X
Cache	F						*	A	A	A	F						*	A	A	A	F						*	A
Com		R	D	-	-	-						R	D	-	-	-						R	D	-	-	-		
Datos						D	D	D	D						D	D	D	D							D	D	D	D

Manteniendo el controlador avanzado con política de página abierta (open-page), ahora la cache de datos implementa además un mecanismo de **prebúsqueda en caso de fallo (prefetch on miss)**. Bajo esta política, ante un fallo de lectura, la cache solicita a la memoria DDR tanto el bloque que ha causado el fallo como el bloque consecutivo.

- d) **Rellena** el siguiente cronograma para el controlador (open-page) con prefetch on miss. Asume que para el primer acceso al vector $\mathbf{v}$, la página de la DDR ya se encuentra abierta.

CLK																												
Ciclo	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
CPU	X							X	X	X	X	X	X	X	X						X	X	X	X	X	X	X	X
Cache	F						*	A	A	A	A	A	A	A	F				*	A	A	A	A	A	A	A	A	A
Com		R	D	-	-	R	D	-	-	-						R	D	-	-	R	D	-	-	-				
Datos						D	D	D	D	D	D								D	D	D	D	D	D				

[illegible][illegible]

Problema 2. (2.5 puntos)

En una **CPU** ejecutamos un programa (X) que realiza 10^9 accesos a datos. Esta **CPU** está conectada a una cache de datos **\$D** con políticas de escritura **copy back + write allocate**. La siguiente tabla muestra algunos datos obtenidos al ejecutar el programa X:

Característica	\$D
Tasa de fallos (m)	10%
Consumo de energía en caso de acierto (Ea)	4 nJ
Penalización en consumo de energía en caso de fallo al reemplazar un bloque no modificado (Ep _f)	20 nJ
Penalización en consumo de energía en caso de fallo al reemplazar un bloque modificado (Ep _{fM})	40 nJ
Porcentaje de bloques modificados (pm)	25%

a) **Calcula** la energía consumida por los accesos a datos del programa X (da el resultado en Joules).

$$E = \text{Accesos} * (E_a + m * (p_m * E_{pfM} + (1 - p_m) * E_{pf})) =$$

$$= 10^9 \text{ a} * (4 \text{ nJ/a} + 0,1 \text{ f/a} * (0,25 * 40 \text{ nJ/fallo} + 0,75 * 20 \text{ nJ/fallo})) = 6,5 \text{ Joules}$$

Sabemos que la cache de datos **\$D** es 2-asociativa y que el acceso a una vía consume 2 nJ. Se desea añadir un predictor de vía a la cache. El predictor de vía seleccionado tiene una tasa de aciertos del 80% para el programa X. El consumo del predictor de vía es insignificante.

b) **Calcula** la energía ahorrada por los accesos a datos del programa X gracias al predictor de vía (da el resultado en Joules).

80% acierto predictor -> 1 via, ahorramos 2 nJ
 10% fallo predictor - acierto cache -> 2 vias no ahorramos nada
 10% fallo predictor - fallo de cache -> 2 vias no ahorramos nada
 $E = 10^9 \cdot a \cdot 0.8 \cdot \text{ap}/a \cdot 2 \text{ nJ}/\text{ap} = 1,6 \text{ Joules}$

Dado el siguiente fragmento de código:

```
for (i=0; i<N; i++)
    suma = suma + v[i]; // v[i] es un vector de doubles (8 bytes)
```

El código está almacenado en la cache de instrucciones \$I (por lo que no provoca fallos), las variables *i*, *N* y *suma* están en registros y \$D está inicialmente vacía. Los elementos del vector *v* son de 8 bytes y los bloques de \$D son de 64 bytes. La capacidad de \$D es de 8 Kbytes.

Hemos ejecutado 2 veces consecutivas el mismo fragmento de código (para *N* = 1000) y hemos medido los ciclos de CPU de ambas ejecuciones:

- En la 1a ejecución el bucle tarda 55.000 ciclos.
- En la 2a ejecución el bucle tarda 30.000 ciclos.

c) **Calcula** el tiempo de penalización medio (en ciclos) en caso de fallo en \$D.

1a ejecución: Cada 8 iteraciones 1 fallo -> 125 fallos
 2a ejecución: reusa el vector -> no hay fallos
 25000 ciclos penalización / 125 fallos = 200 ciclos penalización/fallo

Deseamos ejecutar una sola copia del mismo fragmento de código para *N* muy grande (el vector recorrido es mucho mayor que el tamaño de cache).

d) **Calcula** en función de *N* los ciclos que tarda el fragmento de código anterior.

$30N + 200/8 * N = 55N$ ciclos
 Alt:
 55000 ciclos / 1000 iteraciones = 55 ciclos/iteración
 solo localidad espacial -> Para *N* iteraciones -> $55 * N$ ciclos

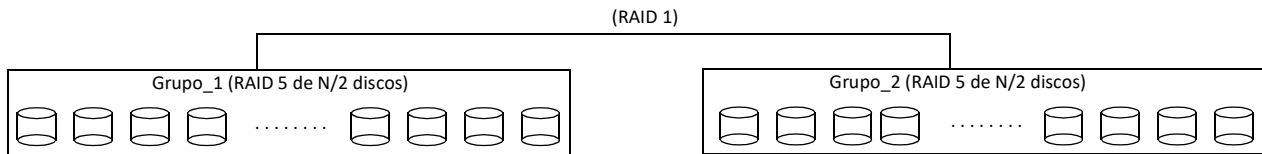
A la cache \$D le añadimos un mecanismo de *prefetch* hardware. Cuando se accede un bloque (*i*) se desencadena *prefetch* del bloque siguiente (*i*+1) siempre que el bloque (*i*+1) no se encuentre ya en la cache o no haya un *prefetch* previo del bloque (*i*+1) pendiente de completar (en ambos casos es innecesario hacer *prefetch* de nuevo).

e) **Calcula** el número máximo de ciclos que puede durar un *prefetch* para que el bucle se ejecute en $40 * N$ ciclos.

Se hace un prefetch cada 8 iteraciones
 $40 \text{ ciclos/iteración} * 8 \text{ iteraciones/prefetch} = 320 \text{ ciclos/prefetch}$

Disponemos ahora de un máximo de **N** discos físicos y vamos a deducir las expresiones del tiempo medio hasta fallo ó MTTF para discos lógicos montados con las siguientes opciones:

- RAID 5 (simple paridad cíclica con $N/2$ discos)
- RAID 51 (mirror doble con 2 grupos RAID 5 de $N/2$ discos por grupo). Ver figura:



Llamaremos **MTTF_d** al tiempo medio hasta fallo de un disco, y **MTTR** al tiempo medio invertido en cambiar un disco averiado y reconstruir la información.

- c) **Escribe** la expresión general para el MTTF del primer sistema: MTTF_RAID 5 (simple paridad cíclica con $N/2$ discos). Debes dar la expresión en función de N , $MTTF_d$, y $MTTR$ cuyos valores son desconocidos.

$$MTTF_RAID5 = MTTF_d / (N/2) \times MTTF_d / (N/2 - 1) \times MTTR = MTTF_d^2 / ((N/2) \times (N/2 - 1) \times MTTR)$$

En RAID5 una vez que falla un disco, el sistema deja de funcionar cuando falla un segundo disco durante el tiempo de recuperación del disco averiado (MTTR)

(ver slides 38 y 39 de 080_Almacenamiento.pdf)

La expresión general para el MTTF del segundo sistema: MTTF_RAID 51 (mirror doble con 2 grupos RAID 5 de $N/2$ discos) la daremos en base al cálculo de MTTF por grupos y es la que se indica con la siguiente expresión:

$$MTTF_RAID\ 51 = \frac{MTTF_grupo1 \times MTTF_grupo2}{2 \times 1 \times MTTR}$$

- d) **Calcula** el valor del MTTF de un sistema RAID51 tal y como indica la anterior expresión. Asume que **N=50** y que los tiempos para **MTTF_d** y **MTTR** son de 75.000 y 25 horas, respectivamente. Indica con claridad los cálculos que realizas para dar tu respuesta.

En RAID51 una vez que falla un grupo, el sistema deja de funcionar cuando falla su segundo grupo durante el tiempo de recuperación del grupo averiado (MTTR)

1 Grupo = RAID5 con 25 discos ($N/2=25$)

$$MTTF_grupo = MTTF_d / 25 \times MTTF_d / 24 \times MTTR = MTTF_d^2 / (25 \times 24 \times MTTR)$$

MTTF_RAID51=

$$MTTF_d^4 / (25^2 \times 24^2 \times 1 \times MTTR^3) = 75000^4 / (25^5 \times 24^2 \times 2) = 2.812.500.000 \text{ horas}$$

[illegible]

Problema 4. (2.6 puntos).

Los procesadores CISC de la familia x86 traducen internamente las instrucciones de lenguaje máquina x86 a microoperaciones que llamaremos “μops”. En una posible implementación de x86 tenemos las siguientes μops:

Tipo	Descripción	Ejemplo
LOAD	Rd <-- M[Rb+Ri*s+despl]	LOAD %R1 <-- M[%EAX+%R2*4-37]
STORE	M[Rb+Ri*s+despl] <-- Rf	STORE M[%EAX+100] <-- %EBX
MOV	Rd <-- Rf Rd <-- inm	MOV %R3 <-- %ESI MOV %EAX <-- \$5
ARITMÉTICO - LÓGICAS	Rd <-- Rf1(op)Rf2 Rd <-- Rf1(op)inm	ADDL %R4 <-- %EAX + %EBX MULL %ECX <-- %R4 * \$38
COMPARACIÓN	Flags <-- Rf1-Rf2 Flags <-- Rf1-inm	CMPL %EAX, %EBX CMPL %R1, \$12
SALTO	Los mismos de x86	JNE loop

Sólo las μ ops LOAD y STORE pueden acceder a memoria, usando los mismos modos de direccionamiento de x86. El resto de μ ops sólo pueden usar registros o inmediatos como operandos. Además de los registros visibles de x86, disponemos de registros adicionales para almacenar valores intermedios: %R0, %R1, ..., %R7.

- a) **Traduce** la secuencia de instrucciones x86 del bucle de la función “scale()”, que se muestra a continuación, a la correspondiente secuencia de microoperaciones. **Calcula** el número de instrucciones dinámicas y microoperaciones en función del valor inicial de %esi.

```
for:
    cmpl    $1, %esi
    jl      endfor

    movl    %edi, %eax
    mull    (%ebx, %esi, 4), %eax
    movl    %eax, (%ebx, %esi, 4)

    decl    %esi
    jmp     for

endifor:
```

Nº instrucciones dinámicas: $7 * \%esi + 2$

```
for:
    cmpl    $1, %esi
    jnl     endfor

    movl    %eax <-- %edi
    load    %r1 <-- M[%ebx+%esi*4]
    mull    %eax <-- %eax*%r1
    store   M(%ebx+%esi*4) <-- %eax

    decl    %esi <-- %esi - $1
    jmp     for

endifor:

Nº microoperaciones:  $8 * \text{\%esi} + 2$ 
```

Se ha evaluado el rendimiento de la función scale() en un centro de datos formado por servidores equipados con multiprocesadores "Beta" que tienen las siguientes características:

Multiprocesador	Voltaje (core)	Corriente fuga (core)	Capacidad equivalente (core)	Frecuencia máxima (core)	Nº cores
Beta	1.0 V	1.6 A	2.0 nF	4.0 GHz	64

El bucle del código ensamblador de scale() se ha ejecutado en el procesador Beta, utilizando un único core a la frecuencia máxima soportada. Se ha medido un IPC medio de 2 y un tiempo de ejecución de 1.75 segundos.

b) **Calcula** los ciclos por microoperación (CP_{μop}) del multiprocesador Beta.

$$CP_{\mu op} = (7 \text{ i} / 8 \text{ o}) * (1 \text{ c} / 2 \text{ i}) = 0.44 \text{ c/o}$$

c) **Calcula** el valor del registro %esi justo antes de ejecutar el bucle de la función scale().

$$n = 2 \text{ i/c} * 1.75 \text{ s} * 4.0 * 10^9 \text{ c/s} = 14 * 10^9 \text{ i}$$

$$\%esi = 14 * 10^9 \text{ i} / (7 \text{ i/iter}) = 2 * 10^9$$

Ahora analizaremos el benchmark STREAM completo que se compone de una primera fase de inicialización seguida de la llamada a la función scale() del apartado a). Por un lado, la fase de inicialización es completamente secuencial y la ejecuta un core a máxima frecuencia. Por otro lado, la función scale() tiene paralelismo a nivel de threads (TLP) y se ejecuta en todos los cores.

Hemos detectado que el multiprocesador Beta disminuye el voltaje a 0.8 V y la frecuencia a 2 GHz de todos los cores durante la fase paralela debido a restricciones térmicas. También sabemos que se siguen produciendo fugas de corriente en aquellos cores que no ejecutan instrucciones por el simple hecho de estar alimentados.

d) **Calcula** las potencias asociadas a la parte secuencial y paralela del benchmark STREAM ejecutadas en el multiprocesador Beta.

$$P_{seq} = 2 * 1^2 * 4 + (1 * 1.6) * 64 = 110.4 \text{ W}$$

$$P_{par} = (2 * 0.8^2 * 2 + 0.8 * 1.6) * 64 = 245.76 \text{ W}$$

Aunque paralelicemos completamente la función scale(), existe un tiempo de sincronización adicional que sigue una ley lineal: $a + b * N$, donde N es el número de cores. Hemos medido las constantes $a = 50 \mu s$ y $b = 10 \mu s$ y un tiempo promedio de 10 ms en la fase de inicialización del benchmark.

e) **Calcula** el consumo del multiprocesador Beta al ejecutar el benchmark STREAM completo (fase de inicialización + función scale).

$$t_{par} = 14 * 10^9 \text{ i} / (2.0 \text{ i/c} * 2.0 * 10^9 \text{ c/s} * 64) + (50 + 10 * 64) * 10^{-6} \text{ s} = 54.69 + 0.69 = 55.38 \text{ ms}$$

$$E_{par} = 55.38 * 10^{-3} * 245.76 = 13.61 \text{ J}$$

$$E_{seq} = 10^{-2} * 110.4 = 1.1 \text{ J}$$

$$E = 14.71 \text{ J}$$